

Raritone

Virtual Try-On Fashion Platform

Internship Project Presentation

MEKA HRISHI TEJA CHOWDARY

Project Overview

Introduction

Raritone is a virtual try-on platform designed to enhance online fashion shopping by allowing users to visualize clothing products digitally before making a purchase.

Objectives

- Improve customer shopping experience.
- Reduce product return rates.
- Provide personalized product visualization.
- Integrate AI-driven virtual try-on functionality.

Expected Outcome

A secure and scalable platform that combines backend services, database management, and AI-assisted image processing.



Key Features



Secure Authentication

JWT-based login with role-based access control and encrypted credentials.



Product Catalog

Managed inventory with categories, categories, images, sizes, and real-time availability.



Avatar Creation

Users build personalized body avatars from uploaded photos for accurate fitting.



Avatar Creation

Users build personalized body avatars from uploaded photos for accurate fitting.



Virtual Try-On

AI engine overlays garments onto user avatars with realistic fabric simulation.



AI Image Processing

Deep learning models handle pose estimation, segmentation, and garment warping.

Technology Stack

Backend

Java 17 · Spring Boot 3 · Spring Security · Spring
Spring Data JPA

Database

MySQL 8 · Hibernate ORM · Flyway

Security

JWT Tokens · BCrypt Password
Hashing · CORS Configuration

Dev Tools

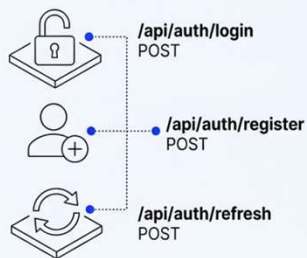
Maven · Git/GitHub · Postman ·

AI Engine

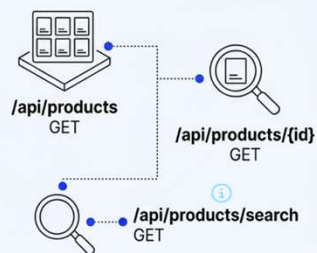
OpenCV · TensorFlow / PyTorch · Custom CNN
Custom CNN Models

REST API Design

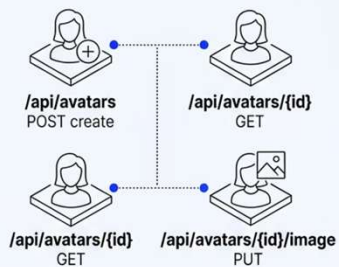
AUTH APIs



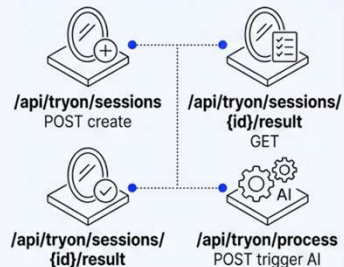
PRODUCT APIs



AVATAR APIs



TRY-ON APIs



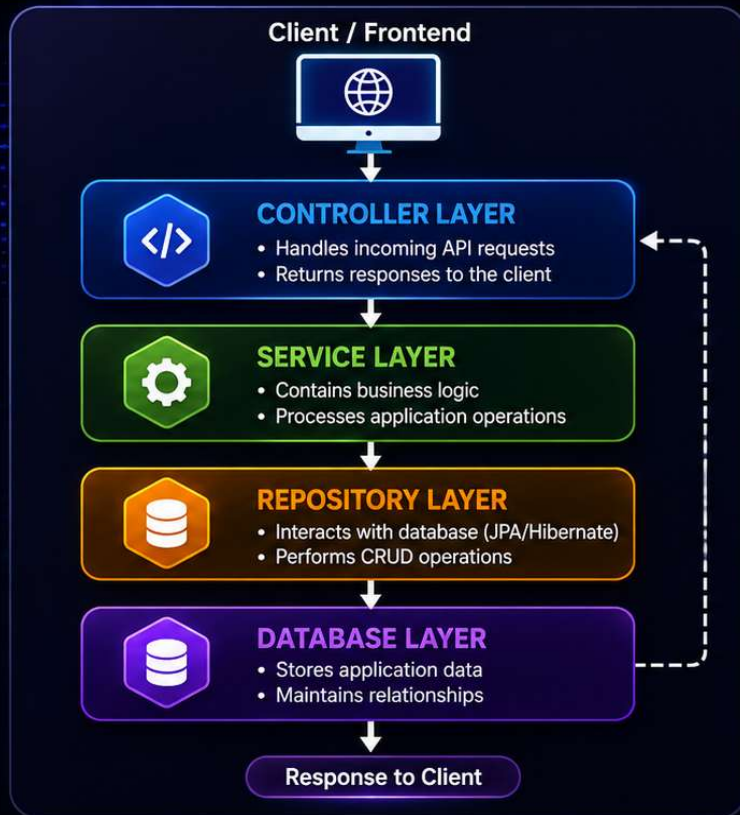
API Standards

- RESTful conventions with proper HTTP status codes
- JSON request/response payloads throughout
- Bearer token authentication on all protected routes
- Pagination and filtering on list endpoints
- OpenAPI/Swagger documentation auto-generated

Sample POST `/api/tryon/process` returns a session ID immediately; results are delivered asynchronously via webhook or polling.

06

BACKEND ARCHITECTURE



Controller Layer

- Handles API requests and responses.
- Acts as the entry point of the application.



Service Layer

- Contains business logic.
- Processes user, product, avatar and try-on operations.



Repository Layer

- Manages database interactions using JPA / Hibernate.
- Performs CRUD operations.



Database Layer

- Stores users, products, avatars and try-on results.
- Ensures data integrity and consistency.

Benefits



Clear separation
of responsibilities



Easier maintenance
and debugging



Scalable and
modular design

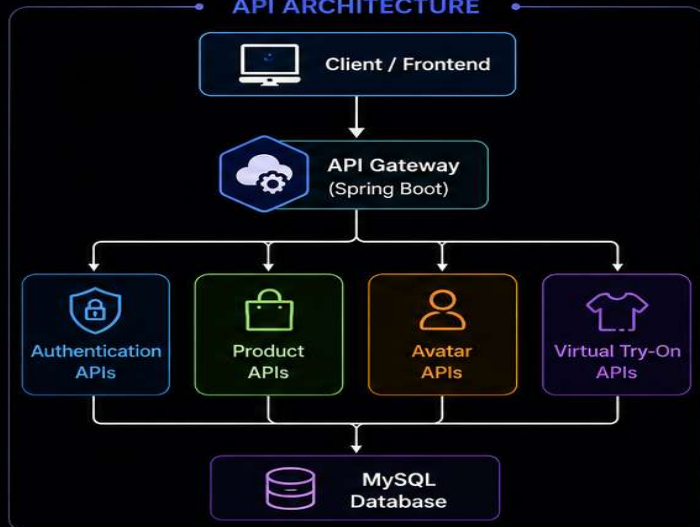


Reusability of code
and components

07 API DOCUMENTATION

The backend exposes RESTful APIs that enable communication between the frontend, database and virtual try-on services.

API ARCHITECTURE



API FEATURES



API ENDPOINTS OVERVIEW

Category	Method	Endpoint	Description	Auth
Authentication APIs	POST	/api/auth/register	Register a new user	No
	POST	/api/auth/login	Login user & get token	No
	POST	/api/auth/refresh	Refresh JWT token	Yes
Product APIs	GET	/api/products	Get all products	Yes
	GET	/api/products/{id}	Get product details	Yes
	GET	/api/products/search	Search products	Yes
Avatar APIs	POST	/api/avatars	Create avatar	Yes
	PUT	/api/avatars/{id}	Update avatar	Yes
	GET	/api/avatars/{id}	Get avatar details	Yes
	DELETE	/api/avatars/{id}	Delete avatar	Yes
Virtual Try-On APIs	POST	/api/tryon	Submit try-on request	Yes
	GET	/api/tryon/{id}	Get try-on result	Yes
	GET	/api/tryon/history	Get try-on history	Yes

AUTHENTICATION FLOW (JWT)



Note: All protected endpoints require a valid JWT token in the Authorization header as:

Authorization: Bearer <token>

Server.js file

```
1  server.js > ...
2  const express = require("express");
3  const dotenv = require("dotenv");
4  const connectDB = require("../config/db");
5
6  dotenv.config();
7
8  // Connect MongoDB
9  connectDB();
10
11 const app = express();
12
13 // Middleware
14 app.use(express.json());
15
16 // Make uploads folder publicly accessible (for images)
17 app.use("/uploads", express.static("uploads"));
18
19 // Routes
20 const productRoutes = require("../routes/productRoutes");
21 app.use("/api/products", productRoutes);
22
23 // Home route
24 app.get("/", (req, res) => {
25   res.send("Server Running Successfully");
26 });
27
28 // Start server
29 const PORT = process.env.PORT || 5000;
30
31 app.listen(PORT, () => {
32   console.log(`Server running on port ${PORT}`);
33 });
34
```

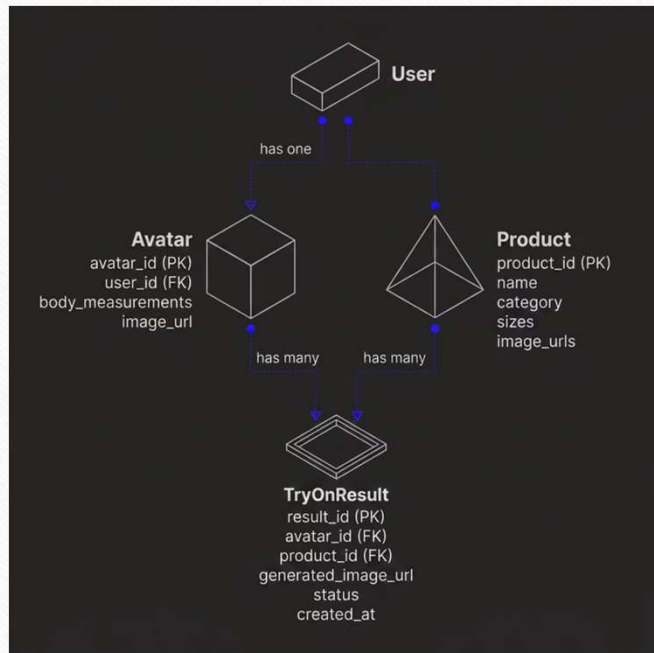
Product.js

```
1  models > .js product.js > productSchema
2  const mongoose = require("mongoose");
3
4  const productSchema = new mongoose.Schema({
5    productName: {
6      type: String,
7      required: true,
8    },
9    brand: {
10     type: String,
11   },
12   category: {
13     type: String,
14   },
15   price: {
16     type: Number,
17     required: true,
18   },
19   stock: {
20     type: Number,
21     default: 0,
22   },
23   image: {
24     type: String,
25   },
26 });
27
28 module.exports = mongoose.model("Product", productSchema);
29
```

ProductController.js

```
1  controllers > .js productcontroller.js > ...
2  const Product = require("../models/product");
3
4  const addProduct = async (req, res) => {
5    try {
6      const product = new Product({
7        productName: req.body.productName,
8        brand: req.body.brand,
9        category: req.body.category,
10       price: req.body.price,
11       stock: req.body.stock,
12       image: req.file ? req.file.filename : "",
13     });
14     await product.save();
15     res.status(201).json({
16       message: "Product Added Successfully",
17       product,
18     });
19   } catch (error) {
20     res.status(500).json({
21       message: error.message,
22     });
23   }
24 };
25
26 const getProducts = async (req, res) => {
27   try {
28     const products = await Product.find();
29     res.json(products);
30   } catch (error) {
31     res.status(500).json({
32       message: error.message,
33     });
34   }
35 };
36
37 module.exports = {
38   addProduct,
39   getProducts,
40 };
41
```


Database Design



Core Entities

- **User** — Credentials, profile, and session metadata
- **Avatar** — Body measurements, reference images, personalization data
- **Product** — Catalog items with size charts and garment images
- **TryOnResult** — Stores AI-generated composite images and status

Design Principles

- Normalized to 3NF to eliminate redundancy
- Foreign keys enforce referential integrity
- Indexed columns on email, product_id, and status

Db.js

```
config > JS db.js > ...
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6
7      console.log("MongoDB Connected");
8    } catch (error) {
9      console.log(error);
10
11      process.exit(1);
12    }
13  };
14
15  module.exports = connectDB;
```

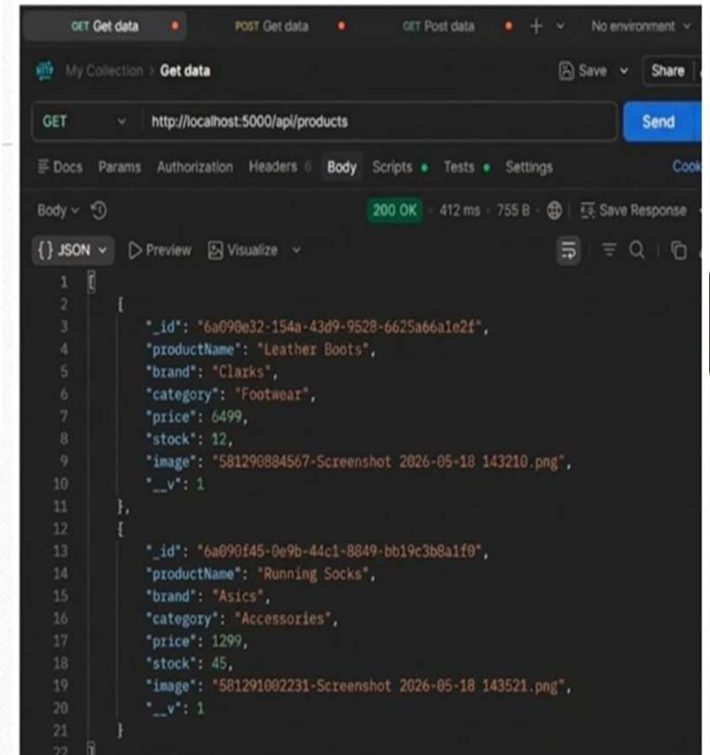
Multer.js

```
config > JS multer.js > ...
1  const multer = require("multer");
2
3  const storage = multer.diskStorage({
4    destination: function (req, file, cb) {
5      cb(null, "uploads/");
6    },
7
8    filename: function (req, file, cb) {
9      cb(null, Date.now() + "-" + file.originalname);
10    },
11  });
12
13  const upload = multer({ storage });
14
15  module.exports = upload;
```

productRoutes.js

```
routes > JS productRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3
4  const {
5    addProduct,
6    getProducts,
7  } = require("../controllers/productController");
8
9  const upload = require("../config/multer");
10
11  router.post("/add", upload.single("image"), addProduct);
12
13  router.get("/", getProducts);
14
15  module.exports = router;
```

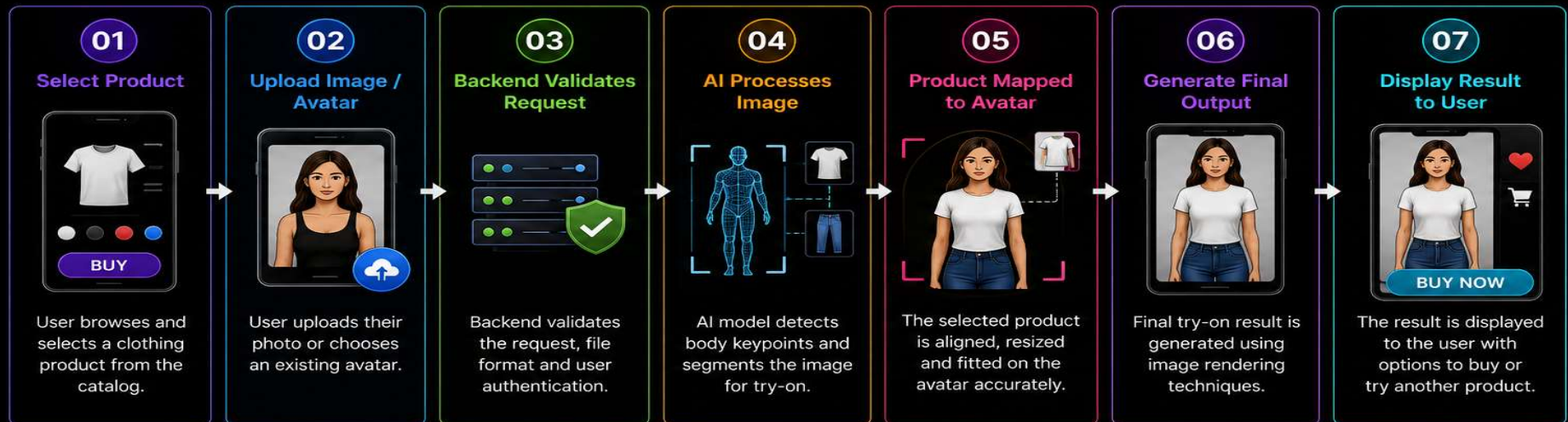
GET API



SLIDE 11

VIRTUAL TRY-ON FLOW

The virtual try-on process allows users to see how a product looks on them using AI-based image processing and accurate product mapping.



KEY BENEFITS



Better Visualization

Users can see how the product looks before purchase.



Increased Confidence

Improves purchase confidence and reduces return rates.



AI-Powered Accuracy

AI ensures precise fitting, alignment and realistic results.



Better Experience

Provides an interactive and personalized shopping experience.



Higher Conversion

Enhances user satisfaction and increases conversion rates.



Outcome: Delivers a realistic and seamless virtual try-on experience, helping users make confident purchase decisions.

Library management system

```
Query 1  SQL File 1* x
Limit to 1000 rows

1 • CREATE DATABASE LibraryDB;
2 • USE LibraryDB;
3 • CREATE TABLE Students (
4     Student_ID INT PRIMARY KEY,
5     Student_Name VARCHAR(50)
6 );
7 • CREATE TABLE Books (
8     Book_ID INT PRIMARY KEY,
9     Book_Name VARCHAR(50)
10 );
11 • CREATE TABLE Borrow (
12     Borrow_ID INT PRIMARY KEY,
13     Student_ID INT,
14     Book_ID INT,
15     FOREIGN KEY (Student_ID)
16     REFERENCES Students(Student_ID),
17     FOREIGN KEY (Book_ID)
18     REFERENCES Books(Book_ID)
19 );
20 • CREATE TABLE Fine (
21     Fine_ID INT PRIMARY KEY,
22     Borrow_ID INT,
23     Amount INT,
24     FOREIGN KEY (Borrow_ID)
25     REFERENCES Borrow(Borrow_ID)
26 );
27 • SHOW TABLES;
```

OUTPUT:

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Result Grid
Tables_in_librarydb				
books				
borrow				
fine				
students				
				Form Editor

USER REGISTRATION API

```
File Edit Selection View ... Q auth
Welcome JS server.js JS db.js JS user.js
JS server.js > ...
1 const express = require("express");
2 const mongoose = require("mongoose");
3 const bcrypt = require("bcryptjs");
4 const app = express();
5 app.use(express.json());
6 mongoose.connect("mongodb://127.0.0.1:27017/backend");
7 const User = mongoose.model("User", {
8   name: String,
9   email: String,
10  password: String
11 });
12 // REGISTER API
13 app.post("/register", async (req, res) => {
14   const { name, email, password } = req.body;
15   if (!name || !email || !password) {
16     return res.json({ message: "All Fields Required" });
17   }
18   if (password.length < 6) {
19     return res.json({ message: "Password Must Be 6 Characters" });
20   }
21   const user = await User.findOne({ email });
22   if (user) {
23     return res.json({ message: "Email Already Exists" });
24   }
25   const hashedPassword = await bcrypt.hash(password, 10);
26   const newUser = new User({
27     name,
28     email,
29     password: hashedPassword
30   });
31   await newUser.save();
32   res.json({ message: "User Registered Successfully" });
33 });
34 // LOGIN API
35 app.post("/login", async (req, res) => {
36   const { email, password } = req.body;
37   const user = await User.findOne({ email });
38   if (!user) {
```

```
39     return res.json({ message: "User Not Found" });
40   }
41   const checkPassword = await bcrypt.compare(password, user.password);
42   if (!checkPassword) {
43     return res.json({ message: "Invalid Password" });
44   }
45   res.json({ message: "Login Successful" });
46 });
47 // GET USERS API
48 app.get("/users", async (req, res) => {
49   const users = await User.find();
50   res.json(users);
51 });
52 // DELETE USER API
53 app.delete("/user/:id", async (req, res) => {
54   await User.findByIdAndDelete(req.params.id);
55   res.json({ message: "User Deleted Successfully" });
56 });
57 app.listen(3000, () => {
58   console.log("Server Running On Port 3000");
59 });
```

```
JS db.js > ...
1 const mongoose = require("mongoose");
2
3 mongoose.connect("mongodb://127.0.0.1:27017/backend");
4
5 console.log("MongoDB Connected");
6
```

```
AUTH Welcome JS server.js JS db.js JS user.js
models > JS user.js > ...
1 const mongoose = require("mongoose");
2
3 const UserSchema = new mongoose.Schema({
4
5   name: String,
6
7   email: String,
8
9   password: String
10 });
11
12 module.exports = mongoose.model("User", UserSchema);
13
```

SLIDE 14

SCALABILITY & FUTURE BACKEND ROADMAP



Goal: Build a highly scalable, reliable and intelligent backend that can handle growing users and provide personalized experiences.

SCALABILITY STRATEGY



1. DATABASE OPTIMIZATION

- Proper indexing and query optimization
- Connection pooling
- Partitioning for large datasets
- Regular database maintenance



2. CACHING

- Redis for frequently accessed data
- Reduce database load
- Improve response time
- Cache strategies for APIs and sessions



3. LOAD BALANCING

- Distribute incoming requests
- Improve availability and reliability
- Prevent server overload
- Horizontal scaling support



4. CLOUD DEPLOYMENT

- Deploy on cloud platforms (AWS, Azure, GCP)
- Auto scaling for traffic spikes
- High availability and disaster recovery
- Cost effective and flexible infrastructure

FUTURE BACKEND ROADMAP



01

Swagger API Documentation

Interactive API documentation for easy testing and integration.



02

Docker Containerization

Containerize the application for consistent and portable deployments across environments.



03

CI/CD Pipelines

Automate build, test and deployment process for faster and reliable releases.



04

Microservices Architecture

Break monolithic application into independent services for better scalability and maintainability.



05

Recommendation Engine

AI/ML based product recommendations for personalized user experience.



06

Real-time Virtual Try-On

Enhance try-on experience with real-time rendering and advanced AI models.

KEY BENEFITS



High Availability



Better Performance



Faster Delivery



Cost Efficient



Future Ready

Security Implementation

JWT Authentication

- Stateless access tokens with short expiry; refresh tokens stored server-side for re-issuance.

Password Encryption

- BCrypt with cost factor 12 — salts generated per user, never stored in plaintext.

Endpoint Protection

- Spring Security filter chain intercepts requests; role-based access restricts admin routes.

Secure Data Handling

- Input validation via Bean Validation, CORS restricted to trusted origins, SQL injection prevented by JPA.



CONCLUSION



PROJECT ACHIEVEMENTS

- ▶ Developed a secure backend using Java and Spring Boot.
- ▶ Designed RESTful APIs for efficient communication.
- ▶ Implemented database management using MySQL.
- ▶ Integrated avatar management and virtual try-on workflows.
- ▶ Applied JWT-based authentication and security.



INTERNSHIP LEARNING OUTCOMES

- ▶ Gained hands-on experience in backend development.
- ▶ Learned API design and integration.
- ▶ Worked with database design and management.
- ▶ Understood scalable system architecture.
- ▶ Improved collaboration using GitHub.



FINAL OUTCOME

Raritone successfully demonstrates how backend technologies, database systems, security mechanisms, and AI-driven processes can be combined to create an intelligent and interactive virtual fashion shopping experience.



HIGH
SECURITY



SCALABLE
ARCHITECTURE



IMPROVED
PERFORMANCE



BETTER
COLLABORATION



FUTURE
READY



THANK YOU